Order & Inventory Sync Service: Design

AI usage note: Used Claude to brainstorm and structure this doc and to check the capacity maths; all decisions were reviewed and are defended by me. *(Edit to reflect what you actually did.)*

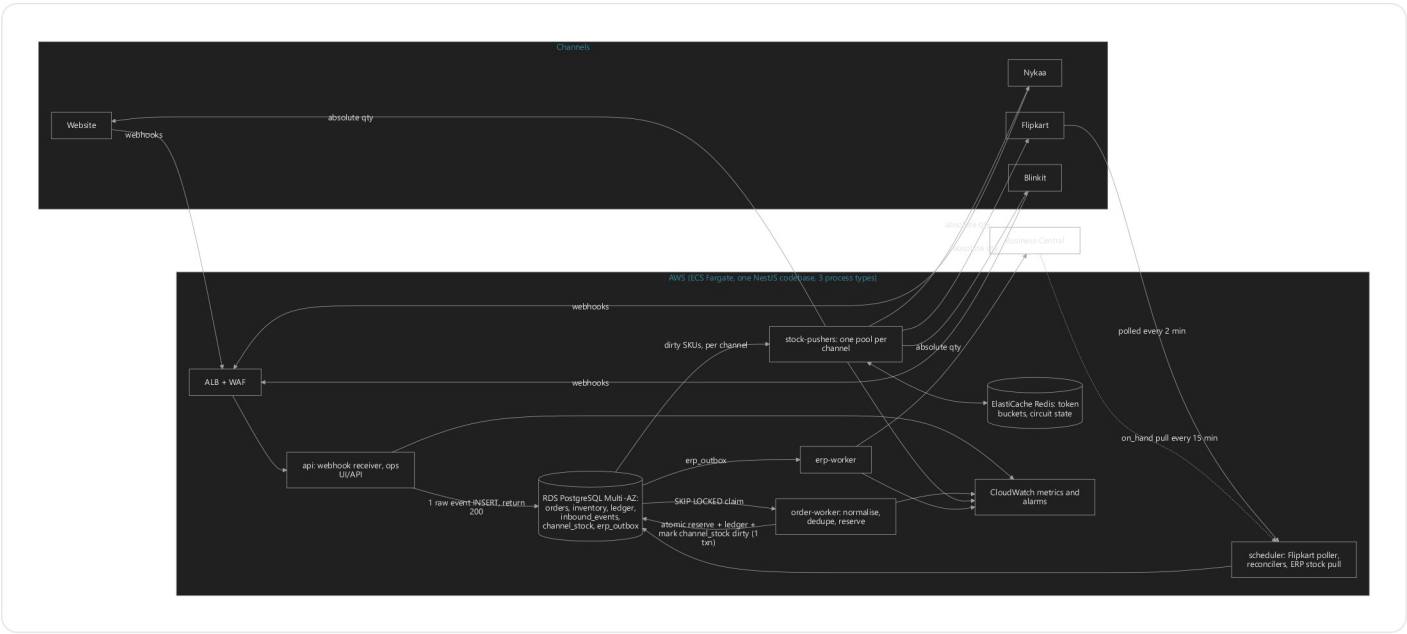
0. Core idea

Marketplace customers have already paid when we hear about the order, so we cannot "reject" a Nykaa/Flipkart order. Overselling is therefore stopped in **two layers**:

1. **Prevention:** show each channel a *conservative* stock number, pushed fast, with the last few units allocated per channel so invisible in-flight orders cannot collide.
2. **Correctness:** one atomic, serialised reservation in Postgres, so stock can never go negative. If prevention fails, exactly one order wins and the loser is flagged immediately.

Postgres is the single source of truth and also the durable queue (transactional outbox). Redis only holds rate-limit tokens, so losing it loses nothing.

1. Architecture



(Export as PNG to `/design/architecture.png` via mermaid.live.)

What runs where: one NestJS modular monolith, deployed as three ECS services from the same image: `api` (2 tasks, behind ALB), `worker` (order, push and ERP consumers, autoscaled on backlog age), `scheduler` (1 task; singleton work guarded by a Postgres advisory lock). RDS Postgres `db.t4g.medium` Multi-AZ, ElastiCache `t4g.small`. No Kafka/SQS: Postgres `SELECT ... FOR UPDATE SKIP LOCKED` is plenty for ~5 orders/s peak.

Cost (approx., verify in AWS calculator): Fargate ~₹6k, RDS Multi-AZ ~₹9k, Redis ~₹2k, ALB+NAT ~₹5k, logs/secrets ~₹3k, which is ≈ **₹25–30k/month**, inside the ₹50k budget.

2. Data model

Table	Key columns	Constraints / purpose
`skus`	id, code, active	`UNIQUE(code)`
`inventory`	sku_id PK, on_hand, reserved, updated_at	`CHECK (reserved >= 0 AND reserved <= on_hand)`. **ATP = on_hand − reserved**
`inventory_ledger` (append-only)	id, sku_id, delta_on_hand, delta_reserved, reason (RESERVE/ RELEASE/DISPATCH/ RETURN_RECEIPT/ERP_ADJUST/ MANUAL), order_id, source_event_id, actor, created_at	`UNIQUE(order_id, sku_id, reason)` makes reservation idempotent. **This is the audit trail**; inventory is a cache of its sum
`inbound_events`	id, channel, external_event_id, payload jsonb, event_time, status, attempts, next_attempt_at, last_error	`UNIQUE(channel, external_event_id)`; raw payload kept for replay
`orders`	id, channel, external_order_id, status, payment_mode, total, event_version	**`UNIQUE(channel, external_order_id)`** is the duplicate-order guard
`order_items`	order_id, sku_id, qty, unit_price	PK(order_id, sku_id)
`channel_stock`	(sku_id, channel) PK, desired_qty, pushed_qty, dirty_since, priority	One row per SKU per channel (~4.8k rows). Pushers read dirty rows
`erp_outbox`	order_id UNIQUE, kind (CREATE/ CANCEL), status, attempts, next_attempt_at, erp_doc_no, last_error	Idempotency key = order_id
`channel_cursor`	channel, last_polled_to	Flipkart polling watermark

Duplicates: three layers: (1) `inbound_events` unique key, (2) `orders` unique key, (3) ledger unique key, so replaying an event can never double-reserve.

3. Order ingestion

- **Webhooks (Website, Nykaa, Blinkit):** verify HMAC, do one `INSERT ... ON CONFLICT DO NOTHING` into `inbound_events`, return 200 in <100 ms. All logic is async, so a slow ERP or a burst never causes webhook timeouts.
- **Flipkart polling:** scheduler polls every 2 min with `modified_since = cursor − 5 min` (overlap is harmless thanks to dedupe). The cursor only advances after the batch is persisted, so a failed poll just re-reads.

- **Normalisation:** per-channel adapter maps payloads to the common order shape (the sample JSON).
- **Out-of-order:** order status is a forward-only state machine (`CREATED → CONFIRMED → CANCELLED → RETURNED`); an event is applied only if it advances state, else logged and ignored. A `CANCELLED` that arrives before `CREATED` inserts a tombstone order, so the late `CREATED` hits the unique key and is ignored and no stock is ever reserved.
- **Channel down:** per-channel circuit breaker and separate worker pools (bulkhead), so one channel's failures never consume another's capacity. Webhook channels retry on their side (assumption); a catch-up job lists orders for the gap once the channel recovers.

4. Inventory consistency: the last unit

Two channels (say Nykaa and Website) sell the last unit in the same second.

1. Both webhooks land in `inbound_events`. Two workers claim them.
2. Each runs one transaction (pseudo-code below). The conditional `UPDATE ... WHERE on_hand - reserved >= qty` takes a **row lock** on `inventory(sku)`, so the two are serialised by Postgres, not by app code or Redis.
3. Winner: `reserved` increments, ledger row written, `channel_stock.desired_qty` recomputed to 0 and marked dirty **in the same commit**.
4. Loser: `UPDATE` affects 0 rows, so the order is set `UNFULFILLABLE` with a ledger/event record. An alert fires, and the cancel on that channel is queued for ops (auto-cancel is v1.1, see §8).
5. Stock = 0 is the highest push priority and goes to all channels first (§5), closing the exposure window.

```text
BEGIN;
  INSERT INTO orders(...) ON CONFLICT (channel, external_order_id) DO NOTHING RETU
  -- no row returned => duplicate: COMMIT and stop
  FOR item IN order.items ORDER BY sku_id:          -- fixed lock order => no dead
    UPDATE inventory SET reserved = reserved + :qty
     WHERE sku_id = :sku AND on_hand - reserved >= :qty RETURNING on_hand, reserve
    IF no row: ROLLBACK; (new txn) set order UNFULFILLABLE; alert; STOP   -- all-o
    INSERT INTO inventory_ledger(sku_id, delta_reserved, reason, order_id) VALUES
    UPSERT channel_stock(sku, ch) SET desired_qty = publish_qty(sku, ch), dirty_si
  UPDATE orders SET status='CONFIRMED';
  INSERT INTO erp_outbox(order_id, kind) VALUES (:id, 'CREATE');
COMMIT;
```

Prevention (what keeps the 0.5% budget): `publish_qty = max(0, ATP - safety_buffer)`.

- *Normal stock:* every channel sees ATP minus a small buffer.
- *Tail stock (ATP ⩽ T, e.g. 10):* switch to **allocation:** `publish_qty(c) = floor(ATP × share_c)` using 7-day channel sales share, with the sum ⩽ ATP − 1. Flipkart's unseen-

for-2-min orders and Blinkit's slow updates can then never collide with other channels, because they only ever sell their own slice. Slices are recomputed after every change.

- *Website* additionally calls a synchronous `reserve` at checkout, so the customer sees "out of stock" instead of a later cancellation.

Cost of this approach: some last units show as 0 on a channel while another still holds them (a small lost-sale risk), traded for near-zero cancellations.

Returns/cancels: cancel before dispatch → `RELEASE` (reserved −qty) immediately. Return after delivery → `RETURN_RECEIPT` (on_hand +qty) when the warehouse confirms receipt; both push stock back via the same dirty mechanism.

5. Stock push (4 channels, 60 s)

`channel_stock` is a **coalescing buffer**: 100 changes to one SKU become one push of the latest value. Pushes are **absolute quantities, not deltas**, so retries and reordering are safe. Each channel has a pusher pool and a Redis token bucket at ~80% of its limit.

Channel	Method	Capacity vs need
Website	direct DB/API write	no limit; instant
Flipkart	bulk 50 SKUs/call, ⩽8 rps	whole 1,200-SKU catalogue = 24 calls (<5 s). No problem
Nykaa	1 SKU/call at 4 rps	**240 SKUs/min**; the real bottleneck (full catalogue = 5 min)
Blinkit	REST for urgent changes (2 rps, retried), daily CSV as full baseline	API flaky; see §6

Because Nykaa can't carry every change in 60 s on a sale day, the queue is **priority-ordered**: (1) SKUs reaching 0, (2) tail-stock SKUs (ATP ⩽ T), (3) others by `dirty_since`. High-stock SKUs tolerate staleness; the only SKUs where staleness causes overselling are the ones pushed first. Normal-load target: p95 lag < 60 s on all channels. Sale-day: p95 < 60 s for priority 1–2; others best effort. Additionally a low-priority hourly **full re-push** corrects any drift.

6. Failure handling

- **Retries:** exponential backoff with jitter (`next_attempt_at`), max 8 attempts for inbound/push. Failures are stored in the same row (`status`, `last_error`), so the DB is the retry queue and nothing is lost on a crash.
- **429 (Flipkart/others):** honour `Retry-After`, drop the token bucket rate, and retry. No tight loops.
- **Blinkit timeouts (2%):** safe to retry because pushes are absolute values. A timeout is treated as "unknown", so we just resend. After 5 consecutive failures the circuit opens, the channel is marked *degraded*, its safety buffer is raised automatically, and the daily CSV

remains the baseline.

- **ERP, exactly-once effect:** `erp_outbox.order_id` is the idempotency key, sent as Business Central's external document number. On a timeout we **look up by that number before retrying**, so we never create a duplicate invoice. Per-order ordering is enforced (CREATE before CANCEL).
- **ERP Sunday downtime:** the circuit opens on consecutive failures, orders keep reserving and queueing in `erp_outbox`, and nothing is lost. Backlog is ~700 orders (normal day, 2 h) and drains at a capped ~5/s on recovery, so ~2.5 min. The Sunday window is configured as expected maintenance (alert thresholds relaxed).
- **Poison messages:** after max attempts → `status = FAILED`. The ops UI lists all failures (inbound, push, ERP) with payload, error and attempt history, offers a **Retry** button (re-queues with the same idempotency key) and records who retried. An alert fires if FAILED > 0 for 15 min.

7. Observability: day-one alerts

1. **Unfulfillable-order rate** (oversell): alert at >0.3% rolling 1 h (budget is 0.5%).
2. **Stock-push lag p95 per channel** (`now - dirty_since`): warn >60 s, page >180 s.
3. **ERP outbox oldest-pending age:** page >30 min (suppressed in the Sunday window until 3 h).
4. **Ingestion health per channel:** time since last event/poll success; FAILED-row count > 0.
5. **Ledger invariant:** `inventory.on_hand/reserved` ≠ ledger sums (should be always zero; any hit is a bug). Plus dashboard: 429/timeout rate and circuit state per channel.

8. Scope: deliberately out of v1

- **Auto-cancelling oversold orders on marketplaces:** each marketplace's cancel rules and seller-rating penalties differ; v1 queues them for ops with a one-click flow. Low volume expected after prevention.
- **Demand forecasting / auto safety buffers:** v1 uses fixed buffers and 7-day sales share.
- **Multi-warehouse, pricing/catalogue sync, partial returns, order edits:** none are needed to fix the stated problem.
- **Kafka/microservices/CDC:** not justified at 5 orders/s with 3 developers and 6 weeks.
- **Rich ops UI:** a simple server-rendered table with retry; no analytics.

9. Trade-offs

Decision	Chosen	Rejected	Why
Queueing	Postgres outbox + `SKIP LOCKED`	SQS, Kafka, BullMQ-on-Redis	Atomic with the stock write (no dual-write problem), no new component, durable, trivially inspectable by ops. Load (≤5/s) is tiny
Oversell control	Row-level conditional `UPDATE` in Postgres	Redis lock / app-level mutex / optimistic version retries	Correctness lives in the same store as the data; no lock expiry edge cases; retry storms avoided on hot SKUs
Stock publication	Safety buffer and per-channel allocation for tail stock	Show full ATP on all channels	Accepts a few lost sales on last units to protect the 0.5% cancellation target and seller ratings
Push model	Coalesced, absolute qty, priority-ordered	Event per change, delta updates	Respects Nykaa's 5 rps; idempotent under retries/timeouts; spends scarce calls on the SKUs that matter
ERP delivery	At-least-once + idempotency key + lookup-before-retry	Distributed transaction / "exactly once" broker	ERP offers no transactions; this gives exactly-once *effect*
Deployment	Modular monolith, 3 process types	Microservices per channel	3 devs, 6 weeks; isolation achieved via separate pools, queues and circuit breakers

10. Assumptions

- Webhooks are signed (HMAC) and marketplaces retry failed deliveries; Nykaa/Blinkit/Website offer an order-list API usable for catch-up.
- A "confirmed" order = payment captured (prepaid) or COD accepted. Orders are all-or-nothing on stock.
- Business Central is the source of truth for warehouse `on_hand` (receipts, damages); we pull it every 15 min as `ERP_ADJUST` ledger entries. Dispatch and `on_hand` decrement are reconciled from ERP. BC supports an external document number with uniqueness or lookup.
- Channels accept absolute stock values and allow setting 0. Blinkit has no stated limit; we cap at 2 rps. Rate limits in the brief are the contract.
- SKU codes are identical across channels (or an `sku_channel_map` is provided).
- Auth: per-channel API keys in AWS Secrets Manager. Time in UTC internally; `event_time` kept for ordering.
- Peak is 300 orders/min ≈ 5/s ≈ ~10 reservation txns/s; a single Postgres primary handles this with large headroom.

11. Stretch A: second warehouse (Bengaluru)

`inventory` and ledger gain `warehouse_id` (PK becomes `(sku_id, warehouse_id)`). Reservation picks a warehouse by shipping pincode zone (nearest with stock, else split-or-fallback), still via the same conditional `UPDATE`. Channel-published stock becomes the sum of warehouses serviceable for that channel. Allocation, ledger and ERP location code extend naturally; the ERP order payload carries the fulfilling warehouse. The main new complexity is split shipments and cross-warehouse rebalancing, which I would scope out initially.